Machine Problem **X**:
The Final Build: Bringing It All Together

CS 1635/2035 - User Interface Design Methodology

Fall 2026

Overview

This final project is designed to serve as a **portfolio-quality Flutter application**. Students will demonstrate mastery of the fundamental principles introduced in the first half of the course and extend their skills with concepts from the latter half.

Objectives

- Apply the **MVVM architectural pattern** using `Provider` or `Riverpod` (if you would like to try something new!)
- Demonstrate proficiency in **asynchronous programming** and **data binding**.
- Integrate an **online API or cloud data source (Firebase)**.
- Incorporate at least ***two*** advanced topics from the second half of the course:
    1. Animation
    2. Gesture
    3. Hardware interaction (sensors, camera, haptics, etc.)
    4. Localization (two or more locales)
    5. Accessibility with Flutter Semantics (screen reader support)

This project explicitly builds upon MP1 (asynchronous programming), MP2 (MVVM and UI composition), and MP3 (data binding and API support) while adding new layers of polish and complexity.

Functional Requirements

1. Data Source Integration

Each app must connect to one of the following:

- A public **REST or GraphQL API** (e.g., SpaceX, OpenWeather, NASA, NewsAPI, etc.)
- **Firebase / Firestore / Realtime Database / Cloud Functions** if the app focuses on user-generated data, collaboration, or synchronization.

Regardless of data source, apps must include clear loading, error, and empty states.

2. MVVM + Data Binding

Follow the MVVM architecture:

- Models: data representation only.
- ViewModels: business logic and async orchestration.
- Views: declarative widget trees bound to immutable state and explicit commands.

3. Asynchronous Programming and Isolates

Use `async/await`, Streams, and/or background isolates (`compute`) to avoid blocking the UI.

4. Advanced Features (choose *2*)

Meaningfully implement *2* of the following features:

- Animation
- Gesture
- Hardware interaction
- Localization
- Screen reader semantics

5. Navigation and Composition

Employ multi-screen navigation, responsive layout, and consistent theming.

6. Production Polish

Include an app icon, launch screen, consistent theming, color contrast compliance, and accessible text scaling.

Example App Ideas

- **EV Charging Companion:** OpenChargeMap API for nearby chargers, favorites list, map view, and animated charge-status cards; includes gesture swipes, haptics, localization, and Flutter semantics.
- **Campus Fitness Scheduler:** Firebase-backed class list, user enrollment, and haptic feedback for usability.
- **Space Weather Dashboard:** Uses NASAs DONKI or NOAA API to display real-time solar flare, geomagnetic, and aurora forecasts. Includes animated solar activity visualizations, async data updates with refresh indicators, localization for scientific terms, and gesture-based navigation through historical events.
- **World Recipe Explorer:** Uses TheMealDB or Spoonacular API for global recipes. Features filters by region and ingredients, async image and recipe loading, swipe gestures to favorite or skip recipes, animated transitions between recipe cards, and localized ingredient lists and measurement units.
- **Mindful Moments:** Uses Firebase for authentication and data storage, with optional quotes API. Provides async saving and loading of journal entries, gesture-based card flipping between mood logs and quotes, animated mood transitions, localized affirmations, and full screen reader and semantic accessibility support.
- **Flight Price Tracker:** Uses AviationStack or Skyscanner API to track real-time flight prices. Implements async search with live updates, animated flight path illustrations, swipe-to-favorite and notification gestures, localization for currencies and units, and optional hardware integration for vibration alerts on price drops.

Deliverables

1. **GitHub Repository** with meaningful commit history.
2. **README.md** including:
    - App pitch and screenshots/GIFs
    - API(s) or Firebase services used
    - MVVM diagram (Models / ViewModels / Views)
    - Build instructions and dependencies
3. **Testing Evidence:** at least 3 widget tests and 1 ViewModel unit test.
4. **Demo Video (3–5 minutes)** showing both advanced features, error/empty states, and MVVM structure.

Grading Rubric (200-220 pts total)

- **A. Core Architecture & Foundations (70 pts)**
  - MVVM separation of concerns; ViewModels expose immutable state (40)
  - Async correctness and isolate use (if applicable) (30)
- **B. Data Source Integration & UX States (35 pts)**
  - Robust loading/empty/error states (20)
  - Proper data formatting and responsive layout (15)
- **C. n Advanced Features (20 x n pts)**
  - Each advanced feature implemented meaningfully (20 for each feature)
- **D. Quality & UX "Polish" (35 pts)**
  - Visual design, accessibility, theming (15)
  - Testing (10)
  - README (10)
- **E. Presentation (20 pts)**
  - Final video demo showing all features (10)
  - Video quality and clarity (10)

CS 2035 additional requirement: The project must build and run on both Android and iOS devices.

Technical Notes

- **State Management:** Use `Provider` (or `Riverpod`). Keep ViewModels testable and UI-agnostic.
- **Threading:** Offload heavy computation to background isolates using `compute()`.
- **Secrets:** Do not commit API keys. Use environment variables or mock keys.
- **Accessibility (if applicable):** Test with TalkBack or VoiceOver; verify contrast and text scaling.
- **Localization (if applicable):** Implement with `intl` and include screenshots in video of all locales.
- **Firebase-specific apps (if applicable):** Must demonstrate stream-based async updates and clear data binding; document security rules or authentication design.

Submission

- Due: **Sunday, December 6 at 11:59pm.**
- Deliverables: GitHub link, zipped project, documentation, video link.

Final Checklist

☐ MVVM diagram with clear boundaries

☐ Async operations and isolates

☐ Online API or Firebase integration (if applicable)

☐ *2* advanced features implemented

☐ Accessibility & localization tested (if applicable)

☐ Tests pass locally

☐ README and demo video complete